

Python Programming & Tools for Data Science

Lecture 2: Scripting & Git

*Melissa Winstanley
University of Washington
October 8, 2026*



First: a quick survey

From the course website:

- > Schedule
- > today's date
- > "Programming Experience Survey"

Please fill it out! This will help me calibrate next week's lesson.

Shell Scripting

Exercise 3

Course webpage:

<https://uw-msds-python.github.io/>

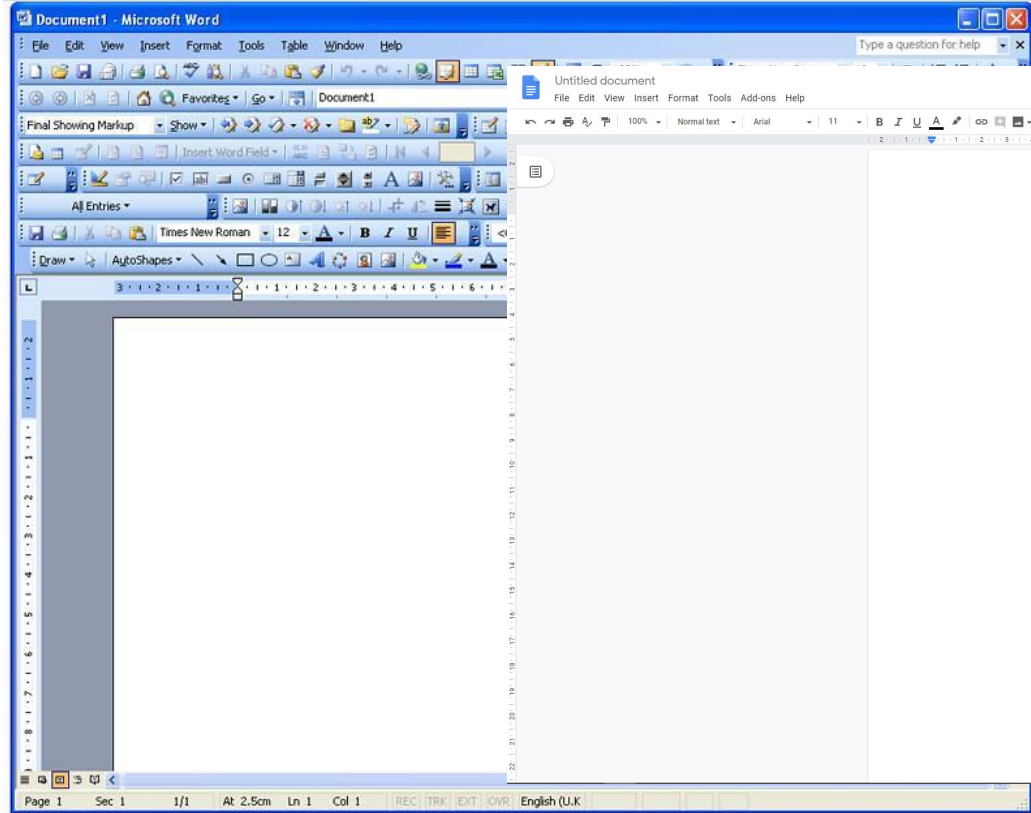
Under “Schedule”, find “Exercise 3”

No need to turn anything in yet - we'll turn in Exercises 3 & 4 together

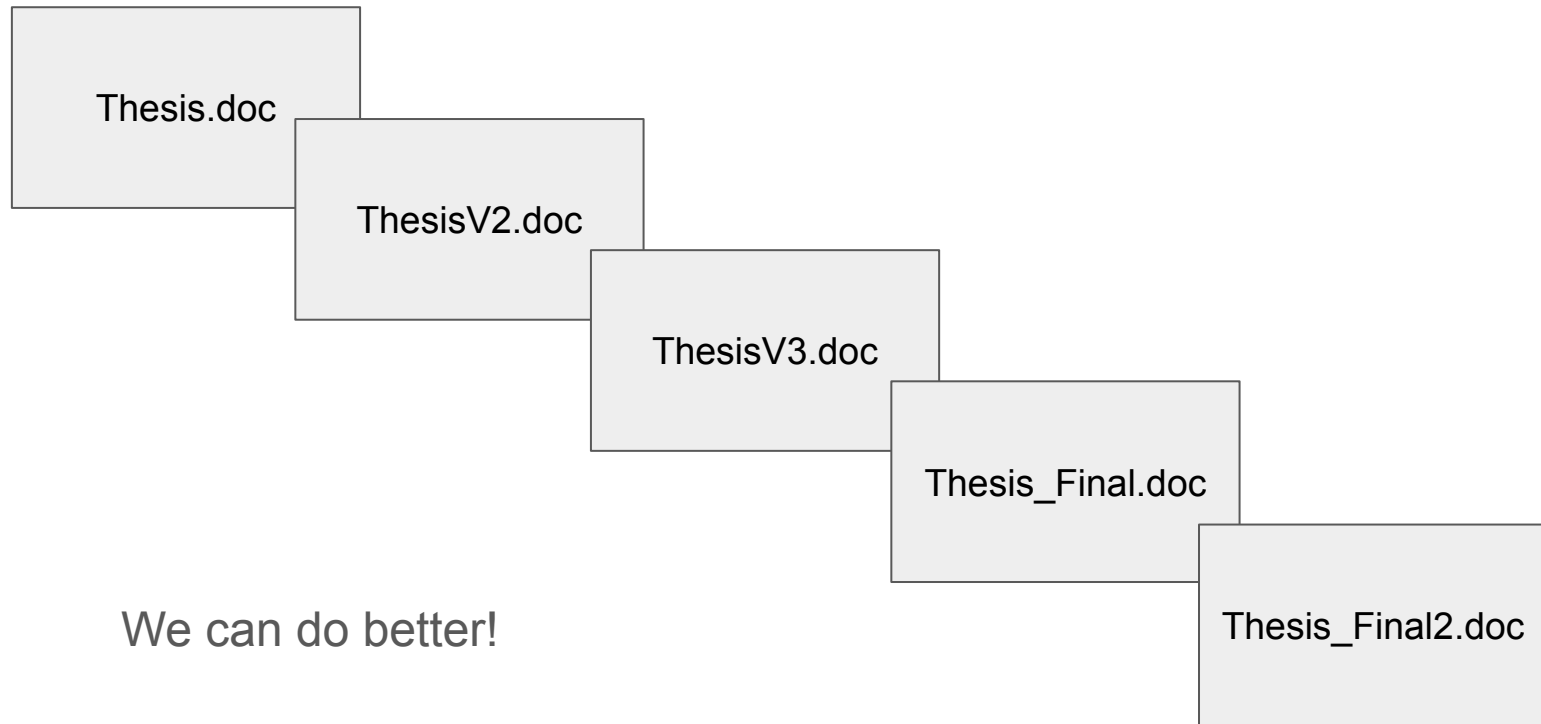
If you need help getting unstuck, and didn't do Exercise 2 last week, there's a key to Exercise 2 on the course website!

Git

Versions in documents



A coarse version control system



What is Version Control for?

- Backups
- Collaboration
 - Do you email files back and forth?
 - Do you save things in a common Google Drive folder?
 - How do you deal with conflicting changes from differing people?
 - How do you keep track of which version is the "final" version?
 - Version control solves these problems
- Version log.
 - Don't worry about whether ctrl-z will work!
 - Keep a log of all previous changes and retrieve those versions whenever you like

Git

In this class, we'll learn about a version control system called **git**, the de facto standard for software development version control (and used widely by industry, academia, etc.)

But, lots of people use version control outside of CS folks, including:

- architects, engineers, and artists making 3D assets (e.g. video games)
- musicians and sound engineers storing different versions of songs
- lawyers keeping track of contract changes over time

Other version control systems: subversion, mercurial, perforce, ...

Repositories (1/2)

A **repository** (or **repo**) in git is a **folder** that stores a copy of all files.

What should be in a repo?

- rule of thumb: everything you need to run the project from source
- source code (e.g. .py files)
- build and configuration files (e.g. scripts)
- assets (e.g. images) & documentation

Repositories (2/2)

What should not be inside of a repository?

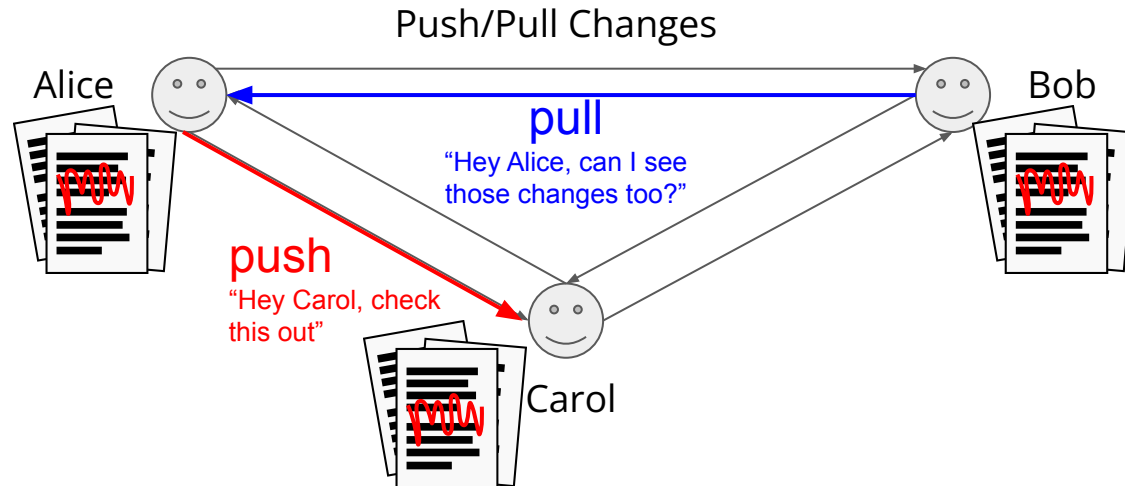
- Rule of thumb: things you can build *from* a source
- For example:
 - executables (e.g. .exe or .app files)
 - cache files (e.g. __pycache__ files, .ipynb_checkpoints)
 - libraries & dependencies
 - often depends on your programming language, build system, and team preference
- Often: data is not inside a repo

Repositories in Git

Git is a “distributed” version control system:

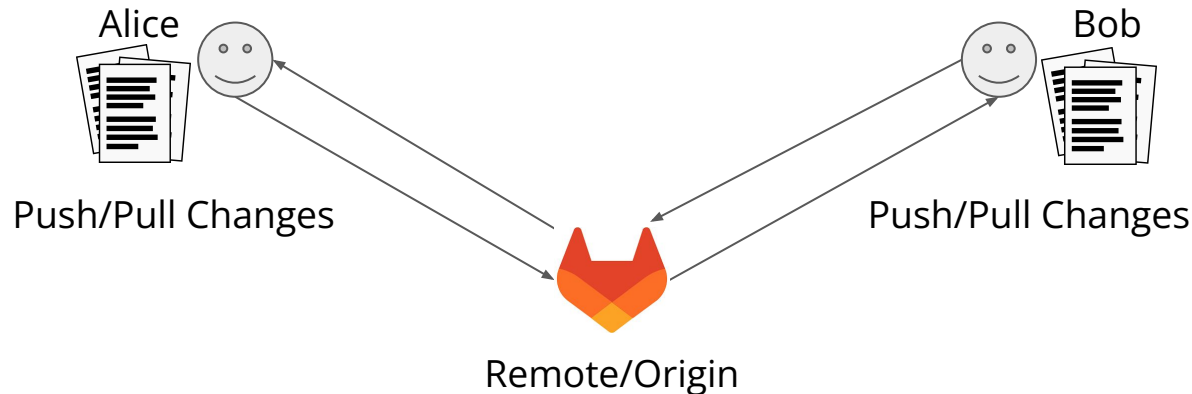
everyone working on the project has a complete version of the repository.

They communicate changes by sending them to each other.



Remote Repositories

However, there is typically a special **remote** repository – a “central” source. Remote is often stored on platforms like GitHub or GitLab; everybody periodically syncs their changes with remote.

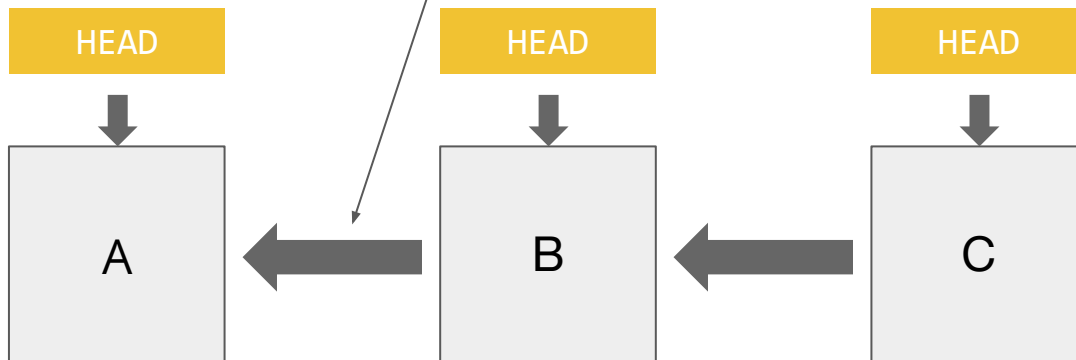


Git actions

- Clone
 - Copy the contents of a git repository onto your computer
- Commit
 - Save a “snapshot” of the current repository
- Push
 - Send updated commits from your computer to the remote
- Pull
 - Copy new commits from the remote to your computer (probably not in this class)

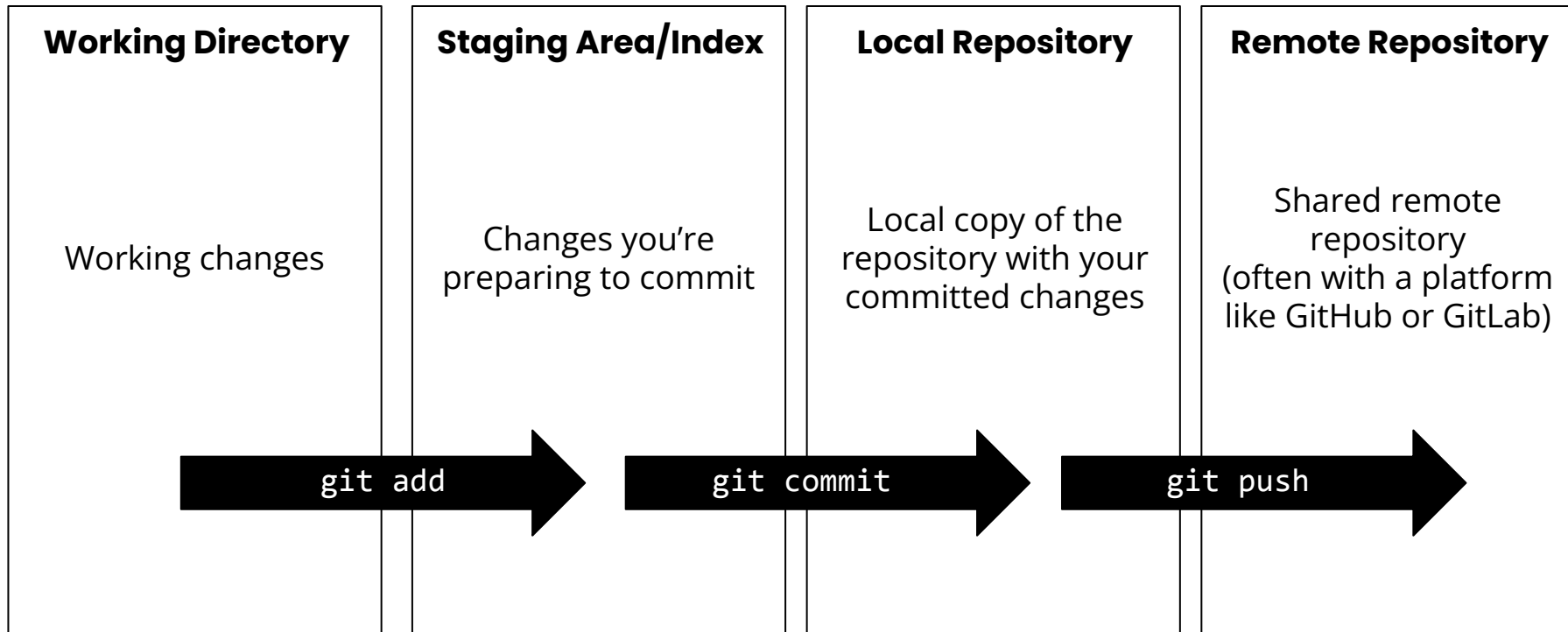
Commits as a list

Each commit references its parent - the one that came before it in time.



These are commits. "A" is the first commit, "C" is the most recent

Git in Four Phases



Demo

Fixing problems

<code>git rm <file></code>	remove a file
<code>git commit --amend</code>	update a commit
<code>git reset --hard HEAD</code>	throw away all working changes, reset to the most recent commit.
<code>git revert HEAD~1</code>	create a NEW commit to undo the commit 1 commit ago

Exercise 4

Course webpage: Under “Schedule”, find “Exercise 4”

Submit via Gradescope once you are done - remember, submit even if you aren't finished!

Git cheat sheet:

```
git clone <repo>
```

```
git status
```

```
git diff
```

```
git add <file>
```

```
git commit      or      git commit -m "Message"
```

```
git push
```